

Resolución 1

Ejercicio 1: Responde verdadero (V) o falso (F)

Dos objetos con el mismo estado pueden tener distinta identidad.	V
Si hay mil objetos de una clase X habrá mil copias de su variable de clase x1.	F
El código: MuertoViviente vampiro ; crea en Java un objeto de la clase MuertoViviente.	F
El código: attr_writer :color crea el consultor y el modificador del atributo color en Ruby.	F
En Ruby los métodos de instancia son públicos y los atributos de instancia son privados, por defecto.	V
La identidad de un objeto en programación orientada a objetos la da su dirección de memoria.	V
Los constructores por defecto devuelven void.	F
La encapsulación de un conjunto de elementos implica de forma implícita su ocultamiento para el resto de elementos del sistema.	F
Cuando definimos una clase, declaramos el estado y definimos el comportamiento de un conjunto de objetos, y en algunas ocasiones también declaramos estado y/o definimos comportamiento de la propia clase.	V
Todos los lenguajes de programación soportan los siguientes atributos de visibilidad de forma explícita para sus atributos: private , package y public	F
Para invocar a los métodos de clase no es necesario que exista previamente una instancia de dicha clase en el sistema.	V
Los paquetes son específicos de Java.	V

→ las u. de clase son = para todos los objetos

→ **fuera new**. sino solo hemos creado un puntero pero sin instancia solo set

→ no devuelve nada.

→ si todos los atrib. son privados si, sino no implica ocultamiento

→ package solo java

Ejercicio 2. Sea la clase java Rectángulo:

```
class Rectangulo {
    float ladoMenor, ladoMayor;
    String color;
    Rectangulo(){
        ladoMenor=2;
        ladoMayor=4;
        color="azul";
    }
    void setColor(String unColor) {
        color = unColor;
    }
    float area(){
        return ladoMenor*ladoMayor;
    }
}
```

Representa lo que va ocurriendo con cada una de estas instrucciones. Se proporciona el resultado de los pasos (1) y (2), haz una figura para cada uno de los pasos (3), (4), (5) y (6).

(1) Rectangulo a, b, c;

(2) a = new Rectangulo();

(3) b = new Rectangulo();
(4) a.setColor("rojo");
(5) float x = a.area();
(6) c = a;

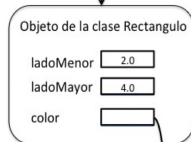
Paso (1)

a
b
c

Paso (2)

a
b
c

Paso 6:
c apunta al mismo lugar que a.



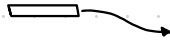
Objeto de la clase String

Paso 4.

paso 8:

a y c queda igual

b



Objeto de la clase Rectangulo

ladoMenor 2.0

ladoMayor 4.0

color



obj. de la clase String

azul

paso 5:

x = 8.0

Ejercicio 3. Dada la clase Java:

```
public class Prueba {
    public static final int A = 1; → atrib. de clase. publica
    static String s = ""; → atrib. de clase. paquete
    private int b = 2; → atrib. de instancia. privado
    int c; → atrib. de instancia. paquete
}
```

a. ¿Cuántos atributos tiene? Indica si se trata de atributos de instancia o de clase. ¿Cuál es la visibilidad de cada uno de ellos?

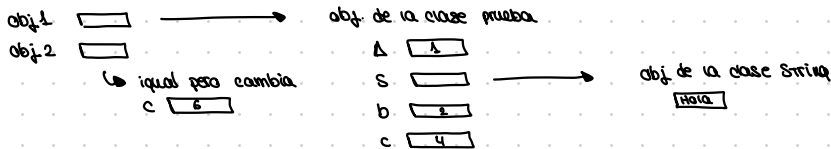
b. A partir del siguiente código, suponiendo que está en otra clase del mismo paquete:

1. Prueba obj1 = new Prueba();
2. Prueba obj2 = new Prueba();
3. obj1.A = 3; → no se puede modificar un final.
4. Prueba.s = "hola";
5. Prueba.A = 14; → no se puede modificar un final.
6. obj1.b = 0;
7. obj2.b = 5;
8. obj1.c = 4;
9. obj2.c = 6;

(En ruby no se puede usar una instancia para acceder a elementos de clase)

b.1. ¿Se produciría algún error de compilación? ¿por qué? (Recomendación: ejecutar el código para comprobarlo).

b.2. Indica cuál es el estado de obj1 y obj2 después de su ejecución y eliminados los errores de compilación si los había.



Ejercicio 4. ¿Quién tiene la responsabilidad de responder a los mensajes que se corresponden con los métodos de clase?

la clase.

Ejercicio 5. Razona si las siguientes afirmaciones son ciertas o falsas, suponiendo que estamos en la clase citada:

1. Los atributos de clase son accesibles sólo desde métodos de clase (no desde métodos de instancia) → *To desde el ámbito de instancia.*
2. Los atributos de instancia son accesibles sólo desde métodos de instancia (no desde métodos de clase). ✓
3. La palabra reservada "this" (Java) puede emplearse tanto en métodos de clase como de instancia. F → *las métodos de clase no tienen un obj. al que referirse.*

Ejercicio 6. ¿Qué mecanismos tiene Java para la ocultación de información? Y ¿Ruby?

Encapsulación

Ejercicio 7. Atributos de instancia y de clase

Sea la siguiente clase en Java:

```
Class C {
    private static int contador2 = 0;
    private int contador1 = 0;
    public void incrementarContador1() {contador1++;}
    public void incrementarContador2() {contador2++;}
    public int getContador1() {return contador1;}
    public int getContador2() {return contador2;}
}
```

Tras ejecutar el siguiente trozo de código:

```
C objeto1 = new C(); → objeto1 = C.new
C objeto2 = new C();
objeto1.incrementarContador1();
objeto1.incrementarContador2();
objeto2.incrementarContador1();
objeto2.incrementarContador2();
int valor1 = objeto1.getContador1();
int valor2 = objeto1.getContador2();
int valor3 = objeto2.getContador1();
int valor4 = objeto2.getContador2();
```

Ruby:

```
class C
  @@contador2 = 0
  def initialize (contra)
    @contador1 = 0
  end
  def incrementarContador1
    contador1 = contador1 + 1
  end
  attr_reader :contador1
```

Se puede hacer sin esto
 def *self*.contador2
 @@contador2

igual pasa cont.2

1. ¿Qué valores tienen valor1, valor2, valor3 y valor4? *valor 1 = valor 3 = 1 valor 2 = valor 4 = 2*
2. Traduce el código Java proporcionado a Ruby

→ *Objeto1 = incrementarContador1*

C.incrementarContador2

valor2 = C.contador2 → sin self si puede usar un obj.

Ejercicio 8. Variables de instancia y de instancia de la clase en Ruby

Sea el siguiente código en Ruby:

```
class A
  @variable = 50  → atrib. de instancia de clase
  def variable
    puts "valor de la variable: #{@variable}" } método de instancia
  end
  def self.variable
    puts "valor de la variable: #{@variable}" } método de clase.
  end
end
```

1. Indica cuál es el resultado de ejecutar las siguientes líneas de código

A.new.variable	crea una instancia de A y llama al método de inst. variable. Devuelve 50.
A.variable	No crea una instancia, llama al método de clase variable. Devuelve 50 independientemente de cuantas instancias haya creadas de A.

2. Explica el porqué de estos resultados

Ejercicio 9. Variables en Ruby

Sea el siguiente código en Ruby:

```
class A
  @@a1=20  → variable de clase
  @a2=7    → variable de instancia clase

  def initialize
    @a3=40  → variable de instancia
  end
  def ver_a1
    puts "valor del atributo a1 #{@a1}"
  end
  def self.ver_a1
    puts "valor del atributo a1 #{@a1}"
  end
  def ver_a2
    puts "valor del atributo a2 #{@a2}"
  end
  def self.ver_a2
    puts "valor del atributo a2 #{@a2}"
  end
  def ver_a3
    puts "valor del atributo a3 #{@a3}"
  end
end
```

Indica cuál es el resultado de ejecutar las siguientes líneas de código en una clase main del mismo módulo:

1. A.new.ver_a1	20. accede al atributo de clase creando una instancia.
2. A.ver_a1	20. accede a través del método de clase. no crea instancia. <i>de instancia</i>
3. A.new.ver_a2	error. no podemos acceder a atrib. de clase desde instancia. <i>error. va a buscar a2 como atrib. de instancia → no hay</i>
4. A.ver_a2	7. puede acceder pq es método de clase.
5. A.new.ver_a3	40. accede a través de una instancia, a una var. de instancia.
6. A.ver_a3	error. método no definido pq no hay self.

Explica el porqué de estos resultados e indica qué tipo de variables son a1, a2 y a3

Ejercicio 10. Paquetes y módulos

Sean los ficheros A.java y B.java cuyo contenido es:

A.java

```
package p1;
import p2.B;
class A { (...) }
```

→ module p1
require_relative 'p2/B'
class A
end

B.java

```
package p2;
public class B {
    String atributo1;
    public String atributo2;
    (...) }
```

→ module p2
class B
def initialize(a1, a2)
 @atributo1 = a1
 ~~@atributo2 = a2~~

attr_accessor :atr2

a) Responde Verdadero (V) o Falso (F)

1. La clase A pertenece al paquete p1

V

2. Como se ha hecho un import de la clase B, atributo1 es accesible desde A

F

3. Como se ha hecho un import de la clase B, atributo2 es accesible desde A

V

Tiene visibilidad de paquete
es público.

b) ¿Es posible traducir exactamente a Ruby el código anterior? En caso negativo, ¿cuál sería la forma más próxima de implementar una estructura similar?

Ejercicio 11. Tipo de variables en Ruby

Sea el siguiente código en Ruby:

```
class Planeta
  @@nombres=['Mercurio','Venus','Tierra','Marte','Júpiter',
    'Saturno','Urano','Neptuno'].freeze
  @@planetas_instanciados = 0

  def initialize
    if @@planetas_instanciados >= @@nombres.size
      raise ArgumentError, 'Lo siento, pero ya se han creado todos
        los planetas, ahora a descansar'
    end
    @nombre = @@nombres[@@planetas_instanciados] var de instancia
    @@planetas_instanciados += 1
    puts "Se ha creado el planeta #{@nombre}!"
  end
end
```

1. Indica cuál es el resultado de ejecutar la siguiente línea de código

1.upto(9) {Planeta.new}

error de 0 a 8 creando una nueva instancia cada vez. Por tanto, crea 8 planetas y en el 9, como no quedan más nombres se ejecuta ArgumentError.

2. Intenta entender el código proporcionado y explica el resultado obtenido

3. Traduce el código anterior a Java

Class Planeta

private static planetas_instanciados;

Package ej;

public class Representante<

private String nom;

private String prenom;

private ArrayList<Client> lista;

public Rep (String n, String p)<

this.nom = n;

lista = new ArrayList<>();

>

public Rep (-----, ArrayList<Client> c)<

lista = c;

module

class Rep.

def initialize (n, p)

@nom = n

@prenom = p

@lista = []

end

def self.new (n, p, c)

@

@

@lista = c

end

def self.new (n, p)

@

@

@

end

private_class_method :new

Ejercicio 12. Ámbito de variables y métodos en Java

Sea el siguiente código en Java:

```
public class Ejercicio12 {
    private static int j = 0;
    private int k = 2;

    public static void muestra_jk() {
        System.out.println("variable clase " + j);
        System.out.println("variable instancia " + k);
    }
    public void muestra_kj() {
        System.out.println("variable instancia " + k);
        System.out.println("variable clase " + j);
    }
    public static void main(String[] args) {
        Ejercicio12.muestra_jk();
        Ejercicio12.j = 10;
        Ejercicio12.muestra_jk();
        Ejercicio12.muestra_kj();
        Ejercicio12 e12 = new Ejercicio12();
        e12.muestra_kj();
        e12.muestra_jk();
    }
}
```

→ Desde un método de clase no se puede acceder a un ámbito de instancia.

→ no se puede llamar sin una instancia, pq no es de clase.

1. Trata de identificar los errores de compilación que hay y elimínalos

2. Ejecútalo y trata de entender qué ocurre

Ejercicio 13. Crea en Java la clase Representante con atributos privados *nombre*, *puesto* y *lista_clientes*, siendo los dos primeros cadenas de texto y el último una lista que contiene objetos de la clase Cliente. Crea un constructor que inicialice los atributos considerando que al crearlo el Representante no tiene clientes.

Crea un nuevo constructor para la misma clase Representante, suponiendo que recibe una lista inicial de clientes para que la inicialice.

Haz el mismo ejercicio en Ruby. Recuerda que no puede haber métodos con igual nombre. ¿Qué soluciones habría para esto? *pp 12-18 12.*

Java:

```
public class Representante {
    private String nombre;
    private String puesto;
    private ArrayList<Cliente> clientes;

    public Representante(String n, String p) {
        nombre = n; // this.nombre = n;
        puesto = p;
        clientes = new ArrayList<>();
    }

    public Representante(String n, String p, ArrayList<Cliente> lista) {
        nombre = n;
        puesto = p;
        clientes = lista;
    }
}
```

Ruby:

```
class Representante
  def inicialize (n, p, a)
    @nombre = n
    @puesto = p
    @cliente = a
  end
  def self.new_simCuentas (n, p)
    new (n, p, [])
  end
  def self.new_conCuentas (n, p,
    new (n, p,
  end
  private_class_method :new # pasa a ser privado.
end.
```

Duda
arrays.
y p9 18.

Ejercicio 14. Sobre el ejercicio 13, supón, que por defecto, el puesto de un representante es el "base" a no ser que se indique en el constructor que es el puesto "avanzado". Escribe el código del constructor en Java y Ruby con la solución a este problema.

Ruby:

```
class Representante
  def inicialize (n, p="base",
    @nombre = n
    @puesto = p

  end
  def self.new_simCuentas (n, p="base")
    new (n, p,
  end
  def self.new_conCuentas (n, p="base",
    new (n, p,
  end
  private_class_method :new # pasa a ser privado.
end.
```

Cadena
" " ?

Java : 2 const. el primero solo recibe nombre y pone "base" → miras char.
el segundo recibe los 3 param.

Ejercicio 15. Comparación de estado e identidad

Dada la siguiente clase:

```
class Comercial{
    private String nombre;
    private Empresa empresa;

    Comercial(String nombre){
        this.nombre=nombre;
        this.empresa=new Empresa("ACME"); //Constructor de Empresa con su nombre
    }
    public void setEmpresa(Empresa empresa){
        this.empresa=empresa;
    }
}
```

`equals` compara estado (atributos)
`==` compara identidad (dirección de memoria)

Y suponiendo que en otra clase se han creado:

```
Comercial c1 = new Comercial("Antonio");
Comercial c2 = new Comercial("Antonio");
Comercial c3 = c1;
Comercial c4 = c1;
```

Indica cuál sería el resultado de las siguientes comparaciones:

<code>c1 == c2;</code>	<code>false</code>
<code>c1 != c2;</code>	<code>true</code>
<code>c1 == c3;</code>	<code>true</code>
<code>c2 == c3;</code>	<code>false</code>
<code>c1.equals(c2);</code>	<code>false</code>
<code>c1.equals(c3);</code>	<code>true</code>
<code>c2.equals(c3);</code>	<code>false</code>
<code>Empresa Refrescos=new Empresa("Refrescos SA");</code> <code>c1.setEmpresa(Refrescos);</code> <code>c1 == c3;</code>	<code>true</code>
<code>c1.equals(c3);</code>	<code>true</code>
<code>c4=null;</code>	<code>nada</code>
<code>c1 == c4;</code>	<code>true</code>

16

17

18

Ejercicio 19. Define en Java y en Ruby una clase cuyas instancias representen atletas y otra clase cuyas instancias sean un equipo de atletas y un entrenador al frente. Incluye los atributos que consideres necesarios. Además, ten en cuenta que necesitamos saber cuántos equipos de atletas hay. Escribe un programa sencillo que cree un equipo y muestre:

- Los atletas que corren en el mismo y su entrenador.
- El número de equipos que tenemos

```
public class Atleta {
    private String nombre;
    private int edad;

    public Atleta (String n, int e) {
        this.nombre = n;
        this.edad = e;
    }

    public String toString() {
        return "Nom: " + nombre + " .... ";
    }
}
```

```
public static void main (String [] args) {
```

```
    Atleta atleta1 = new Atleta("Eva", 22);
    Atleta atleta2 = new Atleta("Juan", 26);
```

```
    Equipo equipo = new Equipo("Maria", "Equipo 1");
    equipo.addAtleta(atleta1);
    equipo.addAtleta(atleta2);
```

```
    System.out.println("info. equipo:");
    equipo.mostrarInformación();
```

```
    System.out.println("Num. de equipos: " + equipo.getNumEquipos());
}
```

```
public class Equipo {
    private String entrenador;
    private ArrayList<Atleta> atletas;
    private String nombre;
    private static int numEquipos = 0;

    public Equipo (String e, String n) {
        this.entrenador = e;
        atletas = new ArrayList<>();
        this.nombre = n;
        numEquipos++;
    }

    public void addAtleta (Atleta a) {
        atletas.add(atleta);
    }

    public static int getNumEquipos() {
        return numEquipos;
    }

    public void mostrarInformación() {
        System.out.println("Nombre equipo: " + nombre);
        (...)
        for (Atleta atleta: atletas) {
            System.out.println("- " + atleta.toString());
        }
    }
}
```

Ruby:

```
class Arrieta
  def initialize (nom, edad)
    @nombre = nom
    @edad = edad
  end
  attr_accessor :numero, :edad

  def to_s
    "Nombre: #{@nombre}, Edad: #{@edad}"
  end
end
```

```
class Equipo
  attr_accessor :nombre, :entrenador, :arrietas
  @@numEquipos

  def initialize (n, e)
    @nombre = n
    @entrenador = e
    @arrietas = []
    @@numEquipos += 1
  end

  def add_arrieta (arrieta)
    @arrietas << arrieta
  end

  def self.numEquipos
    @@numEquipos
  end

  def to_s
    info = "Nombre: #{@nombre}\n"
    info += "Entrenador: ... -\n"
    info += "Arrietas:\n"
    for @arrietas.each do |arrieta|
      info += "  #{@arrieta}\n"
    end
    info
  end
end
```

Programa sencillo:

```
arrieta1 = Arrieta.new (Eva, 22)
arrieta2 = Arrieta.new (Pepe, 28)
entrenador = "Carmen"
```

```
puts "info equipo:"
puts equipo
```

```
equipo = Equipo ("E1", entrenador)
equipo.add_arrieta (arrieta1)
equipo.add_arrieta (arrieta2)
```

```
puts "num equipos: #{@equipo.numEquipos}"
```

si quiero mostrar todos los equipos? :

~~el~~ equipo no es un array
puedo hacer un for equipo do ...